Enterprise Agentic AI for IT Operations Automation

Project Overview & Technical Documentation

Built with LangGraph, FastAPI, React, and Enterprise-Grade Architecture

Author: Ashwin Gurumorthy

Email: gurumurthyashwin@gmail.com

Date: August 2026

Version: 1.0

Table of Contents

1. Project Overview

Enterprise Agentic AI for IT Ops is a production-ready, multi-agent AI platform that automates enterprise IT operations. Built with LangGraph, FastAPI, and React, it uses Large Language Models (LLMs) orchestrated through a state machine to handle incident response, access management, troubleshooting, and knowledge retrieval - all through a natural language chat interface.

Users type requests in plain English, and the system automatically classifies the intent, creates an execution plan, and routes the work to specialized AI agents that carry out tasks using real enterprise tool integrations (Azure AD, Okta, ServiceNow, Jira, GitHub, AWS IAM, Kubernetes, Slack, Microsoft Teams, and more).

2. How It Works - End-to-End Flow

Step 1: User Sends a Message

A user types a request into the React chat interface, such as:

- "My Kubernetes pod is crash looping in production"
- "I need read access to the production database"
- "What's our runbook for handling SSL certificate expiry?"

Step 2: Intent Detection

The IntentDetectionAgent classifies the message using an LLM into one of four intent categories:

Intent	Example
Troubleshooting	Pod is crash looping, Server is unresponsive
Low-Risk Access	Need read access to staging logs
High-Risk Access	Need admin access to production database
General Inquiry	What's our incident response process?

Step 3: Planning

The PlannerAgent creates a step-by-step execution plan with fallback strategies. It defines which agents to invoke, in what order, and what inputs each agent needs.

Step 4: Execution Pipeline

Depending on the detected intent, different agent paths are triggered:

Troubleshooting Flow:

TroubleshootCollect -> SearchKnowledge (RAG) -> AnalyzeRootCause -> ExecuteRemediation -> Notify -> Audit

Access Request Flow:

PolicyAgent (risk eval) -> Auto-Approve or RequestApproval -> AccessManagementAgent -> Notify -> Audit

General Inquiry Flow:

SearchKnowledge (RAG) -> Notify -> Audit

Step 5: Response Delivered

The final response is returned to the user's chat interface, including:

- AI-generated answer with citations from the knowledge base
- Actions taken (access granted, remediation executed, etc.)
- Approval status if human approval was required
- Full audit trail of all operations performed

3. System Architecture

Backend - Python + FastAPI

Component	Technology	Purpose
API Server	FastAPI + Uvicorn	REST API endpoints
Agent Orchestration	LangGraph	State machine for agent workflows
LLM Integration	LangChain + 4 adapters	OpenAI, Anthropic, Gemini, Ollama
Database	PostgreSQL 16	Users, access requests, audit logs
Cache / Memory	Redis 7	4-tier memory system
Vector Database	Qdrant	RAG knowledge base storage
Auth	JWT + RBAC + Azure AD	8 roles, 14 permissions, SSO
Encryption	Fernet (AES)	Secrets encryption at rest
Observability	structlog + OTel + Prometheus	Logging, tracing, metrics

Frontend - React + TypeScript

Component	Technology
Framework	React 18 with TypeScript
Build Tool	Vite 5
Styling	Tailwind CSS 3.4
State Mgmt	Zustand (client) + React Query v5 (server)
Icons	Lucide React
Markdown	React Markdown + Syntax Highlighter

Infrastructure

Layer	Technology
Containerization	Docker + Docker Compose
Orchestration	Kubernetes + Kustomize
IaC	Terraform (AWS: VPC, EKS, RDS, ElastiCache)
CI/CD	GitHub Actions (lint, test, build, deploy)
Cloud	GCP Cloud Run, Render.com

4. The 10 AI Agents

#	Agent	Role
1	IntentDetectionAgent	Classifies user messages into intent categories using LLM
2	PlannerAgent	Creates step-by-step execution plans with fallback strategies
3	TroubleshootingAgent	Collects diagnostics, searches knowledge, diagnoses root causes
4	RAGAgent	Searches knowledge base via vector search with citations
5	PolicyAgent	Evaluates risk scores with configurable thresholds
6	AccessManagementAgent	Processes access requests and executes grants via tools
7	HumanApprovalAgent	Creates/manages approval workflows with expiry and notifications
8	NotificationAgent	Sends alerts via Slack, Microsoft Teams, and Email
9	AuditAgent	Logs every action for compliance and security audit trails
10	MemoryManager	Coordinates the 4-tier memory system across Redis

5. 11 Enterprise Tool Integrations

All tools share a common BaseTool interface with health checks, PII masking, error handling, and structured logging.

#	Tool	Category	Risk	Key Actions
1	Azure AD	Identity	0.8	get_user, list_groups, add_to_group, assign_role
2	Okta	Identity	0.8	get_user, list_apps, assign_app
3	ServiceNow	ITSM	0.3	get_incident, create_incident, search_kb, get_cmdb
4	Jira	Project Mgmt	0.2	get_issue, create_issue, search, add_comment
5	GitHub	Source Code	0.4	get_repo, list_prs, get_pr, list_issues, search_code
6	AWS IAM	Cloud IAM	0.9	list_users, list_roles, attach_policy
7	Kubernetes	Containers	0.85	list_pods, get_pod, scale_deploy, get_logs
8	Slack	Communication	0.1	send_message, list_channels, send_dm
9	MS Teams	Communication	0.1	send_message, list_channels
10	Email	Communication	0.05	send_email via SMTP
11	REST API	Generic	0.3	GET, POST, PUT, DELETE for custom APIs

6. RAG Pipeline (Retrieval-Augmented Generation)

The RAG system allows the AI to search and reference internal documentation:

- Documents (runbooks, SOPs, incident reports) ingested from data/ directory
- DocumentChunker splits into 1000-char chunks with 200-char overlap (sentence-aware)
- EmbeddingService generates vectors using OpenAI text-embedding-3-large or Ollama nomic-embed-text
- Qdrant Vector Store stores embeddings with COSINE similarity search
- On query, top 5 results retrieved (threshold 0.75), LLM generates answer with citations

Knowledge base sources: data/runbooks/, data/sop/, data/incident_reports/, data/knowledge_base/

7. 4-Tier Memory System

Tier	Storage	TTL	Purpose
Session	Redis hash	1 hour	Current conversation context
Conversation	Redis list	24 hours	Full chat history per user
Long-term	Redis hash	30 days	User preferences, learned patterns
Organizational	Redis keys	Permanent	Team policies, common issues, shared knowledge

8. Security Architecture

- JWT Authentication - Access tokens (30 min) + refresh tokens (7 days)
- RBAC - 8 roles (viewer to super_admin) with 14 cumulative permissions
- Azure AD OAuth2 - Enterprise SSO with PKCE flow
- Fernet Encryption - AES encryption for stored secrets and tokens
- PII Masking - Auto detection/masking of emails, phones, SSNs, credit cards, AWS keys
- Audit Logging - Every action logged with user, session, timestamp, risk score
- Rate Limiting - Configurable per-minute request limits
- Security Headers - X-Frame-Options, X-Content-Type-Options, Referrer-Policy

9. LLM Model Routing

The ModelRouter intelligently routes different task types to the most appropriate model:

Task Type	Default Model	Rationale
Intent Detection	GPT-4o Mini	Fast, cheap classification
Planning	GPT-4o	Balanced reasoning
Troubleshooting	GPT-4o	Complex analysis
RAG Query	GPT-4o	Context-aware generation
Policy Evaluation	GPT-4o Mini	Structured risk scoring
Compaction	Same as session	Context compression

Supports 24 pre-configured models across OpenAI, Anthropic, Google, and Ollama with automatic failover and runtime overrides.

10. API Endpoints

All endpoints are prefixed with /api/v1.

Method	Endpoint	Description
POST	/chat/	Send message and trigger agent workflow
GET	/chat/sessions/{id}/history	Retrieve conversation history
POST	/auth/login	Email/password login
POST	/auth/register	User registration
GET	/auth/me	Current user profile
POST	/auth/refresh	Refresh access token
GET	/auth/azure-ad/authorize	Start Azure AD OAuth flow
POST	/access/request	Create an access request
GET	/access/pending	List pending approvals
POST	/access/approve/{id}	Approve or reject a request
GET	/models/	List available LLM models
POST	/models/override	Override task-to-model routing
GET	/drive/auth-url	Get Google Drive OAuth URL
POST	/drive/disconnect	Disconnect Google Drive
GET	/health	Health check
GET	/metrics	Prometheus metrics

11. Database Schema

9 models across 8 tables:

Model	Table	Key Fields
UserModel	users	id (UUID), email, display_name, department, role, teams[]
AccessRequestModel	access_requests	id, user_id, resource_type, access_type, risk_level, status
ApprovalModel	approvals	id, request_id, requester_id, approver_id, status, risk_score
AuditLogModel	audit_logs	id, user_id, session_id, action, resource_type, details (JSONB)
IncidentModel	incidents	id, title, severity, status, affected_services[], root_cause
WorkflowModel	workflows	id, workflow_type, name, risk_level, required_approvals
WorkflowExecModel	workflow_executions	id, workflow_id, session_id, user_id, state
KnowledgeDocModel	knowledge_documents	id, title, content, source, chunk_index, embedding_id
GDriveTokenModel	google_drive_tokens	id, user_id, google_email, access_token, refresh_token

12. Frontend Pages

Page	Description
Chat	Main AI assistant interface with markdown rendering, citations, quick actions
Dashboard	System health, tool status, and operational metrics
Access Requests	Form for requesting access to 9 resource types with Google Drive OAuth
Approvals	Review pending approvals with approve/reject actions and risk badges
Login	Email/password authentication with register/login toggle

13. Deployment Options

- Docker Compose (Local) - 5 services: PostgreSQL, Redis, Qdrant, Backend, Frontend
- Kubernetes - Kustomize overlays for dev, staging, production with health probes
- GCP Cloud Run - Separate Cloud Run services with Supabase, Upstash, Qdrant Cloud ($0/mo free tier)
- AWS (Terraform) - Full infrastructure: VPC, EKS, RDS, ElastiCache, IAM roles
- Render.com - One-click deployment with free-tier PostgreSQL and Redis

14. CI/CD Pipeline

GitHub Actions workflow runs on every push to main/develop:

- Step 1: Lint and Typecheck - ruff check, ruff format, mypy
- Step 2: Test - pytest with PostgreSQL + Redis service containers, coverage to Codecov
- Step 3: Build and Push - Docker build + push to GitHub Container Registry
- Step 4: Deploy - Kubernetes deployment

15. What Has Been Built

Completed

- Full LangGraph state machine workflow with 12 nodes and conditional routing
- 10 AI agents with LLM-powered reasoning
- 11 enterprise tool integrations with common BaseTool interface
- RAG pipeline with Qdrant vector store and document chunking
- 4-tier memory system (session, conversation, long-term, organizational)
- Multi-provider LLM routing with 24 models and automatic failover
- JWT + RBAC + Azure AD OAuth2 authentication
- Risk-based access control with configurable thresholds
- Google Drive integration with OAuth2 and folder sharing
- React frontend with chat, dashboard, access requests, and approvals
- Docker Compose for local development
- Kubernetes manifests with Kustomize overlays
- Terraform configuration for AWS (VPC, EKS, RDS, ElastiCache)
- GitHub Actions CI/CD pipeline
- GCP Cloud Run deployment configuration
- Prometheus metrics and OpenTelemetry tracing
- PII masking on all logs
- Comprehensive security architecture

Areas for Enhancement

- Expand test coverage (unit + integration tests)
- Persist audit logs and approvals to database (currently in-memory)
- Implement streaming responses for chat
- Upgrade password hashing to bcrypt/argon2
- Wire Alembic migrations
- Implement Teams notification (currently a stub)
- Wire knowledge ingestion into the main application

16. Tech Stack Summary

Category	Technology
Backend	Python 3.11, FastAPI, LangGraph, LangChain
Frontend	React 18, TypeScript, Vite 5, Tailwind CSS
LLMs	OpenAI, Anthropic, Google Gemini, Ollama
Database	PostgreSQL 16, Redis 7, Qdrant
Auth	JWT, RBAC, Azure AD OAuth2, Fernet Encryption
Observability	structlog, OpenTelemetry, Prometheus
Infrastructure	Docker, Kubernetes, Terraform, GitHub Actions
Deployment	GCP Cloud Run, AWS EKS, Render.com

-- End of Document --